# Python 3 超基础知识精讲与典型题库

while ·for ·range ·break ·continue ·列表·切片·字符串·元组

**适用对象**　第一次学习 Python 循环与基础序列的学习者。

**题目结构**　10 道单选 + 5 道多选 + 5 道填空 + 4 道大题。

**难度定位**　全部为“识记 + 一步应用 + 简单代码跟踪”，不引入函数、异常、字典或面向对象。

**校验方式**　语义以 Python 官方文档为准；难度参考美国与芬兰大学公开入门课程；全部代码结果已在 Python 3 中实际运行。


原始材料

版本　校订版 1.0

核对日期　2026-07-30

说明

公开题库原题。

# 编制说明

本资料做了三件事：第一，将 6 张图片中的文字、代码、表格与公式重新编码为可搜索、可复制的内容；第二，对手写笔记中的大小写、运算符、切片结果与方法返回值等问题进行技术校订；第三，在同一知识范围内编写 24 道极基础练习，并给出逐项辨析、逐轮跟踪和逐步公式代入。

**准确性层级**

1. 语言语义：以 Python Software Foundation 的官方教程、语言参考和标准库文档为最高依据。
2. 难度校准：参考 MIT 6.100L、Harvard CS50P 与 University of Helsinki Python MOOC 的入门教学顺序。
3. 运行校验：每个题目中的代码输出、列表状态和数值结果均由 Python 3 实际执行验证。

**如何使用**

建议先阅读第 1、2 节，再独立完成第 3 节。核对答案时，不只看选项字母：先用手模拟变量变化，再阅读第 4 节的逐项解释。所有题目均为一星难度，目标是把基本规则掌握牢固，而不是追求复杂技巧。

# 目录

编制说明 I

**1 六页手写笔记的重编码与校订 1**

1.1 流程控制：`while`、`for` 与循环 `else` . . . 1
1.1.1 `while` 循环 . . . 1
1.1.2 `for` 循环 . . . 1
1.1.3 循环 `else` 的准确含义 . . . 2
1.2 `range`：整数序列的边界与步长 . . . 2
1.3 `break` 与 `continue` . . . 3
1.3.1 重写后的教学登录示例 . . . 3
1.4 列表：有序、可重复、可修改 . . . 4
1.4.1 正索引与负索引 . . . 4
1.4.2 切片 . . . 4
1.4.3 常见列表方法 . . . 5
1.5 列表推导式 . . . 5
1.6 字符串：有序、可迭代、不可变 . . . 6
1.7 元组：有序且不可重新绑定位置 . . . 6
1.8 原笔记的关键校订清单 . . . 7

**2 公式、符号与最基础计算 7**

2.1 range 的通项与项数 . . . . . 8
2.2 索引与切片位置 . . . . . 8
2.3 前 $n$ 个正整数之和 . . . . . 8
2.4 偶数和与奇数和 . . . . . 9
2.5 算术平均数 . . . . . 9

**3 超基础典型练习题 9**
3.1 单项选择题（10 题） . . . . . 10
3.2 多项选择题（5 题） . . . . . 12
3.3 填空题（5 题） . . . . . 13
3.4 大题（4 题） . . . . . 13

**4 标准答案与极详细解析 15**
4.1 单项选择题解析 . . . . . 15
4.2 多项选择题解析 . . . . . 19
4.3 填空题解析 . . . . . 21
4.4 大题解析 . . . . . 22

**5 基础自检清单 28**

**6 来源、检索范围与证据说明 28**
6.1 来源使用原则 . . . . . 28
6.2 Python 官方文档 . . . . . 28
6.3 国际大学公开入门课程 . . . . . 29
6.4 质量核验记录 . . . . . 30

# 1 六页手写笔记的重编码与校订

## 1.1 流程控制：while、for 与循环 else

### 1.1.1 while 循环

while 会先对条件表达式做真值判断。结果为真时执行循环体；执行结束后再次判断。结果为假时退出循环。

```
while condition:
    statement_1
    statement_2
else:
    statement_after_natural_finish
```

**三个最基本的检查点**

1. 条件：条件不必本身就是 bool 类型；Python 会判断对象是真值还是假值。例如，非零数字通常为真，零通常为假。
2. 缩进：同一层代码必须保持一致缩进。缩进表示代码块的从属关系。
3. 终止：循环体必须让条件逐步趋向假，或在合适位置使用 break，否则可能形成无限循环。

```
i = 0
while i < 3:
    print(i)
    i += 1
```

上例中，i 依次为 $0, 1, 2$。当 i $= 3$ 时，条件 $3 < 3$ 为假，循环终止。

### 1.1.2 for 循环

for 用于按顺序取出可迭代对象中的元素，并逐个处理：

```
for item in iterable:
    process(item)
else:
    statement_after_natural_finish
```

| | while | for |
|---|---|---|
| 核心控制方式 | 条件保持为真时重复 | 对可迭代对象逐项遍历 |
| 常见用途 | 次数不确定、等待某条件改变 | 处理字符串、列表、元组或 range |
| 终止依据 | 条件变假或执行 break | 遍历完毕或执行 break |

```
message = "Python"
for char in message:
```

```
    print(char)
else:
    print("finished")
```

字符串是可迭代对象，因此变量 char 会依次得到 "P"、"y"、"t"、"h"、"o"、"n"。

### 1.1.3 循环 else 的准确含义

循环后的 else 并非“条件为假时一定执行”的普通条件分支。准确规则是：

- 循环自然结束时，执行 else；
- 循环因 break 提前退出时，不执行 else。

```
for x in range(3):
    print(x)
else:
    print("natural finish")  # runs
```

```
for x in range(3):
    if x == 1:
        break
else:
    print("natural finish")  # does not run
```

依据：S1

## 1.2 range：整数序列的边界与步长

Python 3 中，range(...) 返回的是一个 **range** 对象，不是列表。它表示一个可迭代的整数序列；若要直接看到列表形式，可写 list(range(...))。

| 写法 | 含义 | 示例 |
| --- | --- | --- |
| range(stop) | 起点默认为 0，步长默认为 1 | list(range(4)) 得 [0, 1, 2, 3] |
| range(start, stop) | 从 start 开始，stop 不包含 | list(range(2, 5)) 得 [2, 3, 4] |
| range(start, stop, step) | 每次变化 step，step 可正可负但不可为 0 | list(range(2, 8, 2)) 得 [2, 4, 6] |

若第 $k$ 项从 $k=0$ 开始编号，则候选项可写成

$$a_k = \text{start} + k \cdot \text{step}, \qquad k = 0, 1, 2, \ldots$$

但只有尚未到达或越过 stop 边界的项才会被保留。**stop** 始终不包含。

**一句话记忆**

range 与普通切片都采用“含起点、不含终点”的半开区间思想。

依据：S1、S4

## 1.3 break 与 continue

| 关键字 | 作用 | 之后发生什么 |
| --- | --- | --- |
| break | 立即结束它所在的最内层循环 | 后续轮次不再执行，循环 else 也被跳过 |
| continue | 跳过本轮剩余语句 | 进入同一循环的下一轮 |

```python
for n in range(5):
    if n == 2:
        continue
    if n == 4:
        break
    print(n)
```

执行顺序是 $n=0$ 输出、$n=1$ 输出、$n=2$ 跳过、$n=3$ 输出、$n=4$ 结束，因此输出 $0, 1, 3$。

### 1.3.1 重写后的教学登录示例

原手写示例包含疑似真实格式的账号与密码。本资料不复现这些内容，而使用纯教学占位值：

```python
valid_accounts = [
    ("student", "2468"),
    ("reader", "1357"),
]

while True:
    username = input("Username: ")
    password = input("Password: ")

    if username == "" or password == "":
        print("Input cannot be empty.")
        continue

    if (username, password) in valid_accounts:
        print("Login succeeded.")
        break

    print("Please try again.")
```

此例同时体现：

- Python 关键字必须小写：while、continue；

- 相等比较使用 `==`，赋值使用 `=`；
- `input()` 返回字符串，所以密码与字符串比较；
- “用户名或密码任一为空”使用 `or`；
- 成功时 `break`，空输入时 `continue`。

依据：S2、S6

## 1.4 列表：有序、可重复、可修改

列表可存放不同类型的对象；元素有顺序、可重复，并可按位置修改：

```
items = [4, 7, 16, 32, "A", "Hello", True, None]

print(items[4])  # "A"
items[1] = "B"
del items[2]
```

### 1.4.1 正索引与负索引

设序列长度为 $n$，有效的非负索引为

$$0, 1, 2, \ldots, n-1.$$

负索引 $-1$ 表示最后一项，$-2$ 表示倒数第二项。一般地，

$$-k \longleftrightarrow n-k, \qquad 1 \le k \le n.$$

| 元素 | 54 | 17 | 16 | 17 | 25 | 14.7 |
|---|---|---|---|---|---|---|
| 正索引 | 0 | 1 | 2 | 3 | 4 | 5 |
| 负索引 | -6 | -5 | -4 | -3 | -2 | -1 |

### 1.4.2 切片

切片语法为：

`sequence[start:stop:step]`

其中 `start` 包含，`stop` 不包含，`step` 默认是 1。省略起点时通常从开头开始；省略终点时通常到末尾为止。

```
s = [1, 2, 3, 4, 5]

s[:3]     # [1, 2, 3]
s[1:4]    # [2, 3, 4]
s[::2]    # [1, 3, 5]
s[::-1]   # [5, 4, 3, 2, 1]
```

**索引越界与切片越界的区别**

```
s[100]   # IndexError
s[:100]  # [1, 2, 3, 4, 5]
```

直接索引必须指向一个实际存在的位置；普通切片的边界超出长度时会被安全截断。

### 1.4.3 常见列表方法

| 方法 | 作用 | 关键提醒 |
| --- | --- | --- |
| `append(x)` | 在末尾添加 `x` | 原地修改，正常返回 `None` |
| `insert(i, x)` | 在索引 `i` 之前插入 `x` | 原位置及右侧元素向右移动 |
| `remove(x)` | 删除第一个等于 `x` 的元素 | 找不到时抛出 `ValueError` |
| `pop(i)` | 按索引删除并返回元素 | 省略 `i` 时处理最后一项 |
| `sort()` | 原地排序 | 正常返回 `None` |
| `reverse()` | 原地反转 | 正常返回 `None` |

**混合类型不一定能直接排序**

列表可以混合存放数字、字符串和 `None`，但这不表示这些对象可以互相比较。例如，Python 3 中对 `[1, "A", None]` 直接调用 `sort()` 会抛出 `TypeError`。本资料中的排序题只使用同类、可比较的数字。

依据：S3、S4

## 1.5 列表推导式

列表推导式用于按规则快速构造列表：

```
[expression for variable in iterable]
[expression for variable in iterable if condition]
```

例如，1 到 20 的平方列表：

```
squares = [i ** 2 for i in range(1, 21)]
```

只保留偶数的平方：

```
even_squares = [i ** 2 for i in range(1, 21)
                if i % 2 == 0]
```

可用有序序列记号表示为

$$\left[i^2 \mid i \in \{1, 2, \ldots, 20\},\ i \bmod 2 = 0\right].$$

方括号强调结果是有顺序、允许重复的列表，不是数学集合。

依据：S3、S14

## 1.6 字符串：有序、可迭代、不可变

字符串是字符组成的不可变序列，支持索引与切片，但不能按索引原地改字符：

```
text = "Python"
text[0]       # "P"
text[1:4]     # "yth"
text[0] = "J" # TypeError
```

| 方法 | 返回类型 | 含义 |
| --- | --- | --- |
| `find(sub)` | 整数 | 返回第一次出现的索引；找不到返回 $-1$ |
| `count(sub)` | 整数 | 统计不重叠出现次数 |
| `upper()` / `lower()` | 字符串 | 返回大小写转换后的新字符串 |
| `split(sep)` | 列表 | 按分隔符切分字符串 |
| `strip()` | 字符串 | 返回去掉两端空白后的新字符串 |
| `replace(old,new)` | 字符串 | 返回替换后的新字符串 |
| `startswith(prefix)` | 布尔值 | 判断是否以指定前缀开头 |

**不要说“所有字符串方法都返回新字符串”**

`upper()`、`lower()`、`strip()`、`replace()` 等返回新字符串；但 `find()` 和 `count()` 返回整数，`startswith()` 返回布尔值，`split()` 返回列表。字符串本身不可变，并不意味着每个方法的返回类型都是字符串。

依据：S5

## 1.7 元组：有序且不可重新绑定位置

元组与列表一样有顺序、允许重复、支持索引，也可混合存放不同类型；不同之处是元组的位置不能重新赋值。

```
empty_1 = ()
empty_2 = tuple()
point = (3, 4)
single = (8,)

point[0]     # 3
point.count(3)
point.index(4)
```

**单元素元组为何必须有逗号**

`(8)` 只是带括号的整数 8；`(8,)` 才是只含整数 8 的元组。决定元组语法的是逗号，而不是括号。

**不可变性的精确定义**

元组“不可变”是指元组的槽位不能重新绑定。例如 `t[0] = value` 不允许。但若元组中原本放着一个列表，该列表仍然是可变对象：

```
t = ([1],)
t[0].append(2)  # allowed; t is now ([1, 2],)
```

依据：S3、S4

### 1.8 原笔记的关键校订清单

| 原笔记写法或易误解处 | 校订后 | 原因 |
| --- | --- | --- |
| `While`、`Continue` | `while`、`continue` | Python 关键字区分大小写 |
| 用 `=` 判断相等 | 使用 `==` | `=` 是赋值，`==` 才是相等比较 |
| 密码输入后与整数比较 | 与字符串比较 | `input()` 返回 `str` |
| 用户名和密码均为空才提示 | 任一为空即提示，用 `or` | 需求是“任一个不能为空” |
| 条件表达式结果“必须是布尔值” | 条件会进行真值判断 | 对象不必已经是 `bool` |
| `range`“生成列表” | 返回可迭代的 `range` 对象 | 需用 `list(...)` 显式转为列表 |
| 切片以分号分隔 | `start:stop:step` | Python 切片使用英文冒号 |
| `[1,2,3,4,5][:3]` 得四项 | 正确为 `[1,2,3]` | 终点索引 3 不包含 |
| `Sort()`、`Startswith()` | `sort()`、`startswith()` | 方法名区分大小写 |
| “索引超出范围都会报错” | 直接索引报错，普通切片会截断 | 两种操作的边界规则不同 |
| “字符串方法都返回新字符串” | 返回类型随方法而异 | `find` 返回整数，`split` 返回列表等 |
| 循环 `else` 是普通“否则” | 未被 `break` 中止时才执行 | 它表示循环自然结束 |

## 2 公式、符号与最基础计算

## 2.1 range 的通项与项数

设

$$r = \texttt{range(start, stop, step)}, \qquad \text{step} \neq 0.$$

从第 0 项开始，候选通项为

$$a_k = \text{start} + k \cdot \text{step}, \qquad k = 0, 1, 2, \ldots$$

其中：

- $a_k$：第 $k$ 个候选整数；
- start：起点，包含；
- stop：终止边界，不包含；
- step：每次变化量。

**拓展：项数公式，认识即可**

当 $\text{step} > 0$ 时，

$$N = \max\left(0, \left\lceil \frac{\text{stop} - \text{start}}{\text{step}} \right\rceil\right).$$

当 $\text{step} < 0$ 时，

$$N = \max\left(0, \left\lceil \frac{\text{start} - \text{stop}}{-\text{step}} \right\rceil\right).$$

$\lceil x \rceil$ 表示不小于 $x$ 的最小整数；$\max(0, \cdot)$ 保证项数不会为负。基础练习只需会按“起点、步长、终点不含”逐项列出即可。

## 2.2 索引与切片位置

长度为 $n$ 的序列，其非负索引为

$$0 \le i \le n - 1.$$

负索引 $-k$ 对应非负索引 $n - k$：

$$s[-k] = s[n - k], \qquad 1 \le k \le n.$$

例如 $n = 5$ 时，$s[-1] = s[4]$，$s[-5] = s[0]$。

对步长为 1 的切片 `s[a:b]`，选取的索引为

$$a, a + 1, a + 2, \ldots, b - 1.$$

所以元素个数通常为 $b - a$，前提是 $0 \le a \le b \le n$。

## 2.3 前 $n$ 个正整数之和

当 $n \in \mathbb{Z}_{>0}$ 时，

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

这里：

- $n$ 是最后一个正整数，也是项数；
- $n+1$ 来自首尾配对：$1+n=2+(n-1)=n+1$；
- 除以 2 是因为配对计算中相当于把两份和合在一起。

以 $n=100$ 为例：

$$\frac{100(100+1)}{2}=\frac{100\times 101}{2}=50\times 101=5050.$$

### 2.4 偶数和与奇数和

$$2+4+\cdots+100=2(1+2+\cdots+50)=2\cdot\frac{50\cdot 51}{2}=2550.$$

式中的 50 是 1 到 100 中偶数的个数；把每个偶数提出公因数 2，得到 1 到 50 的和。

前 $m$ 个奇数之和为

$$1+3+\cdots+(2m-1)=m^2.$$

本例 $99=2\times 50-1$，所以

$$1+3+\cdots+99=50^2=2500.$$

### 2.5 算术平均数

对 $n>0$ 个数 $x_1,x_2,\ldots,x_n$，平均数为

$$\bar{x}=\frac{1}{n}\sum_{i=1}^{n}x_i=\frac{x_1+x_2+\cdots+x_n}{n}.$$

其中 $\bar{x}$ 是平均数，$\sum$ 是求和符号，$i$ 是编号，$n$ 是数据个数。

## 3 超基础典型练习题

**作答约定**

- 单选题：每题恰有一个正确选项。
- 多选题：至少有两个正确选项；少选、多选、错选均视为错误。
- 填空题：代码结果按 Python 的表示法书写；字符串可写引号，也可只写内容。
- 所有代码均按 Python 3 解释。

## 3.1 单项选择题（10 题）

### S01 • range 的终点

`list(range(1, 5))` 的结果是：

A. `[1, 2, 3, 4]`

B. `[1, 2, 3, 4, 5]`

C. `[0, 1, 2, 3, 4]`

D. 报错

### S02 • while 的输出

```
i = 0
while i < 3:
    print(i, end=" ")
    i += 1
```

输出是：

A. `0 1 2`

B. `1 2 3`

C. `0 1 2 3`

D. 无限循环

### S03 • break 的输出

```
for i in range(4):
    if i == 2:
        break
    print(i, end=" ")
```

输出是：

A. `0 1`

B. `0 1 2`

C. `2 3`

D. `0 1 3`

### S04 · continue 的输出

```
for i in range(4):
    if i == 1:
        continue
    print(i, end=" ")
```

输出是：

A. `0 1 2 3`
B. `0 2 3`
C. `1`
D. 无输出

### S05 · 负索引

已知 `s = [10, 20, 30]`，表达式 `s[-1]` 的值是：

A. `10`
B. `20`
C. `30`
D. 报错

### S06 · 列表切片

已知 `s = [0, 1, 2, 3, 4]`，`s[1:4]` 的结果是：

A. `[0, 1, 2, 3]`
B. `[1, 2, 3]`
C. `[1, 2, 3, 4]`
D. `[2, 3, 4]`

### S07 · insert

```
a = [1, 3]
a.insert(1, 2)
```

执行后，`a` 是：

A. `[1, 3, 2]`
B. `[2, 1, 3]`
C. `[1, 2, 3]`
D. `[1, 3]`

### S08 · 按索引删除

在索引有效时，哪个列表方法会按索引删除元素，并返回被删除的元素？

A. `append`
B. `remove`
C. `pop`
D. `sort`

#### S09 ·单元素元组

下列哪个 Python 字面量表示“只包含整数 5 的元组”？

A. `5`
B. `(5)`
C. `(5,)`
D. `[5]`

#### S10 ·find

`"python".find("z")` 的返回值是：

A. `0`
B. `-1`
C. `None`
D. 抛出 `ValueError`

### 3.2 多项选择题（5 题）

#### M01 ·range

关于 Python 3 的 `range`，正确的有：

A. `list(range(4)) == [0, 1, 2, 3]`
B. `list(range(2, 7, 2)) == [2, 4, 6]`
C. 步长可以为负数
D. `stop` 一定包含在结果中
E. 步长可以为 0

#### M02 ·列表方法

关于列表方法，正确的有：

A. `append(x)` 会修改原列表
B. `remove(x)` 删除第一个等于 `x` 的元素
C. `pop(i)` 返回被删除的元素
D. `sort()` 返回一个新的已排序列表，原列表不变
E. `reverse()` 返回一个新的逆序列表，原列表不变

#### M03 ·字符串

关于字符串，正确的有：

A. 字符串是不可变序列
B. `upper()` 返回一个新的字符串
C. 无参数 `strip()` 去掉两端空白
D. `replace()` 原地修改原字符串并返回 `None`
E. f-string 可以把表达式结果嵌入字符串

#### M04 ·元组

关于元组，正确的有：

A. `(1,)` 是单元素元组
B. 元组支持索引
C. 不能直接给元组的某个槽位重新赋值
D. 元组中的值可以重复
E. 元组内部的列表会自动变成不可变对象

#### M05 ·控制流

关于 Python 控制流，正确的有：

A. `for` 可以遍历可迭代对象
B. `break` 退出最近一层循环
C. `continue` 进入最近一层循环的下一轮
D. `while False` 的循环体至少执行一次
E. 循环 `else` 只在循环未因 `break` 中止时执行

### 3.3 填空题（5 题）

#### F01 ·带步长的 range

`list(range(3, 8, 2))` 的结果是 _____。

#### F02 ·切片

已知 `items = ["a", "b", "c", "d"]`，`items[1:3]` 的结果是 _____。

#### F03 ·strip

`" Py ".strip()` 的字符串内容是 _____。（可不写引号）

#### F04 ·append 与 len

执行 `a = [1, 2]; a.append(3)` 后，`len(a)` 等于 _____。

#### F05 ·单元素元组

为使 `t = (8\_\_\_)` 成为单元素元组，应填入符号 _____。

### 3.4 大题（4 题）

#### L01 ·最基础累加

用 `for` 和 `range` 计算从 1 到 100 的所有整数之和，并输出结果。要求：

1. 写出完整代码；
2. 写出输出；
3. 使用 $1+2+\cdots+n$ 的求和公式验证。

### L02 ·break、continue 与偶数

阅读代码，写出输出，并用一个列表推导式在本题范围内产生相同结果。

```
result = []

for x in range(1, 10):
    if x > 6:
        break
    if x % 2 == 1:
        continue
    result.append(x)

print(result)
```

### L03 ·列表状态、切片与平均数

执行以下代码，分别写出 data、middle 与 average 的值，并用平均数公式解释 average。

```
data = [4, 1, 3]
data.append(2)
data.sort()
middle = data[1:3]
average = sum(data) / len(data)
```

### L04 ·字符串处理与元组摘要

阅读代码，写出 clean、title、parts、summary 的值，并说明 raw 是否改变。

```
raw = "  python,java,python  "
clean = raw.strip()
title = clean.replace("python", "Python")
parts = title.split(",")
summary = ("Python", parts.count("Python"))
```

# 4 标准答案与极详细解析

**答案速查**

| | |
|---|---|
| 单选 | S01 A；S02 A；S03 A；S04 B；S05 C；S06 B；S07 C；S08 C；S09 C；S10 B |
| 多选 | M01 ABC；M02 ABC；M03 ABCE；M04 ABCD；M05 ABCE |
| 填空 | F01 `[3, 5, 7]`；F02 `["b", "c"]`；F03 `"Py"`；F04 3；F05 逗号 `,` |
| 大题 | L01 5050；L02 `[2, 4, 6]`；L03 `[1,2,3,4]`、`[2,3]`、2.5；L04 `("Python", 2)` |

## 4.1 单项选择题解析

**S01 • 答案 A**

规则拆分： `range(1, 5)` 中，`start=1` 会被包含，`stop=5` 不包含，未写出的 `step` 默认为 1。因此依次得到

$$1, \quad 1+1=2, \quad 2+1=3, \quad 3+1=4.$$

下一个候选数 5 已到达终止边界，不能加入。用 `list(...)` 转换后为 `[1, 2, 3, 4]`。

逐项辨析：

- **A.** 正确。包含起点 1，不包含终点 5。
- **B.** 错误。把 `stop=5` 错误地包含了。
- **C.** 错误。已经显式指定起点为 1，不会从 0 开始。
- **D.** 错误。`start<stop` 且默认正步长，写法合法。

易错点：Python 3 中 `range` 本身不是列表，所以题目特意写了 `list(range(...))`。 依据：S1、S4

**S02 •答案 A**

逐轮跟踪：

| 判断前的 `i` | `i < 3` | 输出 | `i += 1` 后 |
|---|---|---|---|
| 0 | 真 | 0 | 1 |
| 1 | 真 | 1 | 2 |
| 2 | 真 | 2 | 3 |
| 3 | 假 | 不输出 | 循环结束 |

所以输出为 `0 1 2`。末尾空格来自 `end=" "`，通常答题时可不标。
逐项辨析：

- **A.** 正确。`i` 为 $0, 1, 2$ 时条件为真。
- **B.** 错误。第一次输出在自增之前，所以先输出 0，不是 1。
- **C.** 错误。`i` $= 3$ 时 $3 < 3$ 为假。
- **D.** 错误。每轮都有 `i += 1`，变量会到达终止条件。

等价理解：`i += 1` 就是 `i = i + 1`。 依据：S1、S10

**S03 •答案 A**

`range(4)` 依次提供 $0, 1, 2, 3$。

| `i` | `i == 2` | 动作 |
|---|---|---|
| 0 | 假 | 跳过 `break`，输出 0 |
| 1 | 假 | 跳过 `break`，输出 1 |
| 2 | 真 | 立即执行 `break`，不再执行下方 `print` |
| 3 | — | 循环已经结束，没有机会处理 |

因此输出为 `0 1`。

- **A.** 正确。只输出 `break` 发生前的 $0, 1$。
- **B.** 错误。`i` $= 2$ 时先判断再 `break`，所以 2 不输出。
- **C.** 错误。循环从 0 开始，而且 2 触发终止。
- **D.** 错误。`break` 终止整个当前循环，3 不会继续执行。

关键顺序：若把 `print` 放到 `break` 判断之前，输出会改变；所以跟踪代码时必须尊重语句顺序。
依据：S1、S2

**S04 •答案 B**

| `i` | `i == 1` | 动作 |
|---|---|---|
| 0 | 假 | 输出 0 |
| 1 | 真 | 执行 `continue`，本轮 `print` 被跳过 |
| 2 | 假 | 输出 2 |
| 3 | 假 | 输出 3 |

所以输出是 `0 2 3`。

- **A.** 错误。1 所在轮次的输出被跳过。
- **B.** 正确。只跳过 1，后续 2, 3 仍会处理。
- **C.** 错误。恰好是 1 不输出。
- **D.** 错误。代码在多个轮次都会到达 `print`。

核心区别：`continue` 只结束本轮，`break` 才结束整个当前循环。 依据：S1、S2

**S05 •答案 C**

列表长度 $n = 3$。负索引 $-k$ 对应非负索引 $n - k$。本题 $k = 1$，因此

$$n - k = 3 - 1 = 2, \qquad \texttt{s[-1]} = \texttt{s[2]} = 30.$$

- **A.** 错误。10 的索引是 0 或 $-3$。
- **B.** 错误。20 的索引是 1 或 $-2$。
- **C.** 正确。$-1$ 是最后一个元素。
- **D.** 错误。对长度为 3 的列表，$-1$ 是合法索引。

易错点：$-1$ 不是“非法的负位置”，而是从右向左计数的第一个位置。 依据：S4

**S06 •答案 B**

切片 `s[1:4]` 的 `start=1`、`stop=4`、默认 `step=1`。选中的索引为

$$1,\ 2,\ 3,$$

对应的元素为 `[1, 2, 3]`。

- **A.** 错误。索引 0 在起点之前。
- **B.** 正确。包含索引 1，不包含索引 4。
- **C.** 错误。元素 4 位于终止索引 4，不包含。
- **D.** 错误。起始位置本身会被包含，不是从下一个位置开始。

检验公式：在 $0 \le 1 \le 4 \le 5$ 且步长为 1 时，元素个数为 $4 - 1 = 3$。 依据：S4

### S07 •答案 C

`insert(i, x)` 表示把 `x` 放到索引 `i` 处；原来位于该位置及其右侧的元素向右移动：

$$[1,\ 3] \xrightarrow{\texttt{insert(1, 2)}} [1,\ 2,\ 3].$$

- **A.** 错误。这是把 2 加到末尾的效果，更像 `append(2)`。
- **B.** 错误。若要放到开头，应使用 `insert(0, 2)`。
- **C.** 正确。元素 2 成为索引 1 处的新元素。
- **D.** 错误。`insert` 会原地修改列表。

参数次序：第一个参数是位置，第二个参数是要插入的值。 依据：S3

### S08 •答案 C

例：

```
s = [10, 20, 30]
x = s.pop(1)
```

执行后 `s == [10, 30]`，且 `x == 20`。因此答案是 **pop**。

- **A.** 错误。`append(x)` 在末尾添加元素。
- **B.** 错误。`remove(x)` 按值删除第一个匹配项，而且正常返回 `None`。
- **C.** 正确。`pop(i)` 按索引删除并返回该元素。
- **D.** 错误。`sort()` 用于原地排序。

边界提醒：题干说明“索引有效”。若索引越界，`pop(i)` 会抛出 `IndexError`。 依据：S3

### S09 •答案 C

只有 **`(5,)`** 是只含整数 5 的元组。创建单元素元组的关键是尾随逗号。

- **A.** 错误。这只是整数 5。
- **B.** 错误。括号只起分组作用，`(5)` 仍是整数。
- **C.** 正确。逗号使它成为单元素元组。
- **D.** 错误。方括号创建列表。

类型检验：

```
type((5))   # int
type((5,))  # tuple
```

依据：S3、S4

**S10 •答案 B**

`str.find(sub)` 在找到子串时返回第一次出现的索引，找不到时返回 $-1$。因为字符 `"z"` 不在 `"python"` 中，所以结果为 `-1`。

- **A.** 错误。索引 0 表示在开头找到目标，但开头是 `"p"`。
- **B.** 正确。找不到时 `find` 返回 $-1$。
- **C.** 错误。`find` 不在此处返回 `None`。
- **D.** 错误。`find` 把“找不到”作为正常结果；不要与列表或元组的 `index` 混淆。

依据：S5

## 4.2 多项选择题解析

**M01 •答案 A、B、C**

- **A.** ✓ 正确。`range(4)` 等价于从 0 起、到 4 前停止，得到 $0, 1, 2, 3$。
- **B.** ✓ 正确。从 2 起每次加 2：$2, 4, 6$；下一个 8 已超过终点 7。
- **C.** ✓ 正确。负步长用于递减，例如 `list(range(5, 0, -2)) == [5, 3, 1]`。
- **D.** ✗ 错误。`stop` 不包含。
- **E.** ✗ 错误。步长必须满足 $\text{step} \neq 0$；否则会抛出 `ValueError`。

通项

$$a_k = \text{start} + k \cdot \text{step}$$

只负责产生候选值；正步长时需保持 $a_k < \text{stop}$，负步长时需保持 $a_k > \text{stop}$。 依据：S1、S4

**M02 •答案 A、B、C**

- **A.** ✓ 正确。`append(x)` 在原列表末尾加入 `x`。
- **B.** ✓ 正确。`remove(x)` 只删除第一个等于 `x` 的元素。
- **C.** ✓ 正确。`pop(i)` 删除并返回索引 `i` 处的元素。
- **D.** ✗ 错误。`sort()` 原地排序，正常返回 `None`。
- **E.** ✗ 错误。`reverse()` 原地反转，正常返回 `None`。

**典型错误**

```
numbers = [3, 1, 2]
numbers = numbers.sort()  # wrong: numbers becomes None
```

正确写法是先单独执行 `numbers.sort()`，之后继续使用已被修改的 `numbers`。

依据：S3

### M03 •答案 A、B、C、E

- **A.** ✓ 正确。字符串不能通过索引赋值原地修改。
- **B.** ✓ 正确。`upper()` 返回大写形式的新字符串。
- **C.** ✓ 正确。无参数 `strip()` 去除两端空白，不会自动删掉中间空格。
- **D.** ✗ 错误。`replace()` 返回替换后的新字符串；原字符串不变，返回值也不是 `None`。
- **E.** ✓ 正确。例如 `f"\{1 + 2\}"` 的结果是字符串 `"3"`。

为什么要接收返回值：

```
s = "banana"
s.replace("a", "o")
print(s)  # still "banana"

s = s.replace("a", "o")
print(s)  # "bonono"
```

依据：S5、S7

### M04 •答案 A、B、C、D

- **A.** ✓ 正确。尾随逗号使 `(1,)` 成为单元素元组。
- **B.** ✓ 正确。元组是序列，可以使用正索引和负索引读取。
- **C.** ✓ 正确。元组创建后，不能执行 `t[i] = value`。
- **D.** ✓ 正确。元组允许重复，例如 `(1, 1, 2)` 合法。
- **E.** ✗ 错误。元组不会自动冻结内部的可变对象。

不可变性的边界：

```
t = ([1],)
t[0].append(2)  # allowed: t becomes ([1, 2],)
t[0] = [1, 2]   # TypeError: tuple slot reassignment
```

第一行修改的是内部列表；第二行试图重新绑定元组槽位，二者不同。 依据：S3、S4

### M05 •答案 A、B、C、E

- **A.** ✓ 正确。`for` 可按顺序遍历字符串、列表、元组和 `range` 等可迭代对象。
- **B.** ✓ 正确。嵌套循环中，`break` 只退出它直接所在的最内层循环。
- **C.** ✓ 正确。`continue` 跳过本轮剩余语句，进入最近一层循环的下一轮。
- **D.** ✗ 错误。`while` 在进入循环体之前先判断；`False` 使循环体执行 0 次。
- **E.** ✓ 正确。循环自然结束时执行 `else`；因 `break` 中止时跳过 `else`。

一个容易忽略的细节：`while False: ... else: ...` 中，循环体执行 0 次，但循环没有被 `break` 中止，所以 `else` 会执行。 依据：S1、S2

## 4.3 填空题解析

**F01 ·答案 `[3, 5, 7]`**

起点为 3，步长为 2，终止边界为 8 且不包含。逐项计算：

$$3, \qquad 3+2=5, \qquad 5+2=7, \qquad 7+2=9.$$

9 已超过边界，因此舍去。最终为 `[3, 5, 7]`。 依据：S1、S4

**F02 ·答案 `["b", "c"]`**

| 索引 | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 元素 | "a" | "b" | "c" | "d" |

`items[1:3]` 取索引 $1, 2$，不取索引 3。所以结果是 `["b", "c"]`。 依据：S4

**F03 ·答案 `"Py"`**

`strip()` 默认删除两端的空白字符：

$$\texttt{" Py "} \longrightarrow \texttt{"Py"}.$$

它返回一个新字符串；中间的字符 `P` 与 `y` 不改变。按题目说明，只写 `Py` 也正确。 依据：S5

**F04 ·答案 3**

`append(3)` 后，

$$\texttt{[1, 2]} \longrightarrow \texttt{[1, 2, 3]}.$$

列表中有三个元素，因此 $\mathrm{len}(\texttt{a}) = 3$。这里的 3 表示元素个数；它恰好也与最后一个元素的值相同，但两者含义不同。 依据：S3、S6

**F05 ·答案逗号 `,`**

`(8)` 仍是整数 8；写成 `(8,)` 才是单元素元组。因此空格中应填一个英文逗号。完整结果为

```
t = (8,)
```

依据：S3、S4

## 4.4 大题解析

### L01 •从 1 累加到 100

参考程序：

```
total = 0

for number in range(1, 101):
    total += number

print(total)
```

输出：

```
5050
```

#### 第 1 步：为什么是 `range(1, 101)`

`range(1, 101)` 从 1 开始，并在 101 之前停止，因此遍历

$$1, 2, 3, \ldots, 100.$$

若错误地写成 `range(1, 100)`，最后一个数只会到 99。

#### 第 2 步：为什么从 `total = 0` 开始

0 是加法单位元：对任意数 $x$，都有 $0 + x = x$。因此在还没有加入任何数时，累计和自然应为 0。

#### 第 3 步：每轮究竟做什么

`total += number` 等价于

`total = total + number`.

前几轮和最后一轮如下：

| number | 加法前 total | 运算 | 加法后 total |
|---|---|---|---|
| 1 | 0 | $0 + 1$ | 1 |
| 2 | 1 | $1 + 2$ | 3 |
| 3 | 3 | $3 + 3$ | 6 |
| 4 | 6 | $6 + 4$ | 10 |
| ⋮ | ⋮ | ⋮ | ⋮ |
| 100 | 4950 | $4950 + 100$ | 5050 |

**第 4 步：用公式验证**

前 $n$ 个正整数之和：

$$S_n = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

符号含义：

- $S_n$：从 1 到 $n$ 的总和；
- $n$：最后一项，同时也是项数；
- 本题 $n = 100$。

逐步代入：

$$\begin{aligned} S_{100} &= \frac{100(100+1)}{2} \\ &= \frac{100 \times 101}{2} \\ &= \frac{100}{2} \times 101 \\ &= 50 \times 101 \\ &= 5050. \end{aligned}$$

程序与公式得到同一结果，互相验证。

**三个高频错误**

1. 把终点写成 100，导致没有加到 100；
2. 把 `total = 0` 写在循环内部，导致每轮重新清零；
3. 忘记缩进 `total += number`，导致循环并未累计。

依据：S1、S4、S9

## L02 •break、continue 与偶数

输出：

```
[2, 4, 6]
```

### 第 1 步：读懂两个判断的先后次序

代码先判断

$$x > 6,$$

满足时执行 `break`，整个循环终止。若不满足，再判断

$$x \bmod 2 = 1.$$

mod 表示取余数。整数除以 2 后：

- 余数为 0：偶数；
- 余数为 1：正奇数。

奇数执行 `continue`，因此跳过后面的 `result.append(x)`。

### 第 2 步：逐轮跟踪

| x | $x > 6$ | $x \bmod 2 = 1$ | 动作 | result |
|---|---|---|---|---|
| 1 | 假 | 真 | `continue` | `[]` |
| 2 | 假 | 假 | 加入 2 | `[2]` |
| 3 | 假 | 真 | `continue` | `[2]` |
| 4 | 假 | 假 | 加入 4 | `[2, 4]` |
| 5 | 假 | 真 | `continue` | `[2, 4]` |
| 6 | 假 | 假 | 加入 6 | `[2, 4, 6]` |
| 7 | 真 | 不再判断 | `break` | `[2, 4, 6]` |

执行 `break` 后，$8, 9$ 都不会再处理。

### 第 3 步：用列表推导式产生相同结果

```
result = [x for x in range(1, 7)
          if x % 2 == 0]
```

结构对应：

$$\underbrace{x}_{\text{加入列表的值}} \quad \underbrace{\text{for } x \text{ in range}(1,7)}_{\text{只遍历1 到6}} \quad \underbrace{\text{if } x \bmod 2 = 0}_{\text{只保留偶数}}.$$

因此保留 $2, 4, 6$。

**“相同结果”不等于一般语义完全等价**

列表推导式没有 `break`。这里先把原循环真正会保留的范围收缩为 `range(1, 7)`，所以在本题输入范围内得到相同列表；不能把它推广为所有带 `break` 循环的一般等价改写。

依据：S1、S2、S14

## L03 ·列表状态、切片与平均数

最终答案：

```
data == [1, 2, 3, 4]
middle == [2, 3]
average == 2.5
```

### 第 1 步：append

初始列表是

$$\texttt{[4, 1, 3]}.$$

执行 `data.append(2)` 后，2 被放到末尾：

$$\texttt{[4, 1, 3, 2]}.$$

### 第 2 步：sort

`data.sort()` 对原列表进行升序排列：

$$\texttt{[4, 1, 3, 2]} \longrightarrow \texttt{[1, 2, 3, 4]}.$$

`sort()` 原地修改列表，正常返回 `None`。代码没有把返回值赋回 `data`，所以写法正确。

### 第 3 步：切片

$$\texttt{middle = data[1:3]}.$$

选中的索引为 $1, 2$，不含 3。在排序后的列表中：

| 索引 | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 元素 | 1 | 2 | 3 | 4 |

因此

$$\texttt{middle == [2, 3]}.$$

**第 4 步：计算全部四个元素的平均数**

题目使用 `sum(data)` 和 `len(data)`，所以平均数针对完整的 **`data`**，不是 `middle`。
算术平均数公式：

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

各符号含义：

- $\bar{x}$：平均数；
- $n$：数据个数，本题 $n = 4$；
- $x_i$：第 $i$ 个数据；
- $\sum$：将所有指定数据相加。

代入 $x_1 = 1, x_2 = 2, x_3 = 3, x_4 = 4$：

$$\begin{aligned}\bar{x} &= \frac{1+2+3+4}{4} \\ &= \frac{10}{4} \\ &= 2.5.\end{aligned}$$

程序对应关系为：

```python
sum(data)  # 10
len(data)  # 4
10 / 4     # 2.5
```

依据：S3、S4、S11

## L04 •字符串处理与元组摘要

最终答案：

```
clean == "python,java,python"
title == "Python,java,Python"
parts == ["Python", "java", "Python"]
summary == ("Python", 2)
raw == "  python,java,python  "
```

**第 1 步：strip**

```
clean = raw.strip()
```

`strip()` 去掉两端空白，不删除中间字符：

$$\texttt{" python,java,python "} \longrightarrow \texttt{"python,java,python"}.$$

**第 2 步：replace**

```
title = clean.replace("python", "Python")
```

两个完全匹配的 "python" 都被替换：

$$\text{"python,java,python"} \longrightarrow \text{"Python,java,Python"}.$$

replace() 返回新字符串，clean 本身不被原地修改。

**第 3 步：split**

```
parts = title.split(",")
```

逗号是分隔符，分隔符本身不进入结果：

$$\text{"Python,java,Python"} \longrightarrow [\text{"Python"}, \text{"java"}, \text{"Python"}].$$

split 的结果类型是列表。

**第 4 步：计数并建立元组**

```
parts.count("Python")  # 2
```

列表中 "Python" 出现两次，因此

$$\text{summary} = (\text{"Python"}, 2).$$

这是一个二元素元组：第一个元素是字符串，第二个元素是整数。元组可以存放不同类型。

**第 5 步：raw 是否改变**

没有改变。字符串不可变；strip() 和 replace() 都把结果放进了新变量。因此原始值仍是

$$\text{" python,java,python "}.$$

**返回类型总复盘**

| 表达式 | 返回类型 | 本题返回值 |
|---|---|---|
| raw.strip() | str | "python,java,python" |
| clean.replace(...) | str | "Python,java,Python" |
| title.split(",") | list | ["Python","java","Python"] |
| parts.count(...) | int | 2 |

依据：S5、S12、S13

# 5 基础自检清单

完成全部题目后，应能不看答案判断下列陈述：

| | 陈述 | 应答 |
|---|---|---|
| 1 | `range` 包含起点但不包含终点。 | 正确 |
| 2 | `while` 条件必须已经是 `bool` 类型。 | 错误；会做真值判断 |
| 3 | `break` 只退出最近一层循环。 | 正确 |
| 4 | `continue` 会终止整个循环。 | 错误；只跳过本轮 |
| 5 | 循环因 `break` 结束时仍会执行循环 `else`。 | 错误 |
| 6 | 直接索引越界与普通切片越界的行为相同。 | 错误 |
| 7 | `remove(x)` 按值删，`pop(i)` 按索引删并返回。 | 正确 |
| 8 | `sort()` 返回新的已排序列表。 | 错误；原地修改并返回 `None` |
| 9 | 字符串不可变，但字符串方法的返回类型不全是字符串。 | 正确 |
| 10 | 单元素元组必须写出逗号。 | 正确 |

# 6 来源、检索范围与证据说明

## 6.1 来源使用原则

1. 规则确认：优先使用 Python Software Foundation 官方文档，不以论坛、博客或题库作为语言语义的最终依据。
2. 全球难度校准：使用美国 MIT、Harvard 与芬兰 University of Helsinki 的公开入门课程，确认所选知识点属于初学阶段，并保持例题只做最基础应用。
3. 原创性：24 道题由本资料按图片知识范围重新编写；公开来源只用于核验语义、范围和难度，未复制其题干。
4. 检索与核对日期：2026-07-30。网页内容可能更新，因此保留了机构、标题与直接链接。

## 6.2 Python 官方文档

| ID | 支持范围 | 直接链接 |
| --- | --- | --- |
| S1 | 控制流、`range`、`break`/`continue`、循环 `else` | Python Tutorial: More Control Flow Tools |
| S2 | `break` 与 `continue` 的精确定义 | Python Language Reference: Simple statements |
| S3 | 列表方法、列表推导式、`del`、元组 | Python Tutorial: Data Structures |
| S4 | 序列、索引、切片、`range`、元组 | Python Standard Library: Built-in Types |
| S5 | 字符串方法 | Python Standard Library: String Methods |
| S6 | `input`、`len`、`list`、`tuple`、`range`、`sum` | Python Standard Library: Built-in Functions |
| S7 | f-string | Python Tutorial: Formatted string literals |

## 6.3 国际大学公开入门课程

| ID | 机构与用途 | 直接链接 |
| --- | --- | --- |
| S8 | MIT OpenCourseWare（美国）：入门顺序，覆盖字符串、迭代、元组、列表、可变性与推导式 | MIT 6.100L: Material by Lecture |
| S9 | Harvard University（美国）：循环基础示例 | CS50P Lecture 2: Loops |
| S10 | University of Helsinki（芬兰）：`while True`、计数与 `break` | Python Programming MOOC: Simple loops |
| S11 | University of Helsinki（芬兰）：列表索引、可变性、增删与排序 | Python Programming MOOC: Lists |
| S12 | University of Helsinki（芬兰）：字符串不可变、切片、`count`/`replace` | Python Programming MOOC: Strings and lists |
| S13 | University of Helsinki（芬兰）：元组索引、不可变性、与列表的区别 | Python Programming MOOC: Tuple |

| ID | 机构与用途 | 直接链接 |
| --- | --- | --- |
| S14 | University of Helsinki（芬兰）：列表推导式与筛选条件 | Python Programming MOOC: List comprehensions |

## 6.4 质量核验记录

| 检查项 | 方法 | 结果 |
| --- | --- | --- |
| 题型数量 | 自动统计：单选 10、多选 5、填空 5、大题 4 | 通过 |
| 单选唯一性 | 独立审校每个选项及完整语句顺序 | 通过 |
| 多选答案 | 逐项按官方语义判断 | 通过 |
| 代码输出 | Python 3 实际运行断言 | 全部通过 |
| 数值公式 | 循环结果与闭式公式互相验证 | 通过 |
| 知识范围 | 仅保留图片中的控制流、列表、字符串、元组 | 通过 |
| 安全处理 | 登录示例仅用教学占位值 | 通过 |

**最终学习目标**

如果你能独立解释“为什么 `stop` 不包含”“`break` 与 `continue` 的区别”“切片为何少取终点”“`sort()` 为什么不能赋回原变量”“字符串为何不原地改变”以及“单元素元组为什么需要逗号”，那么这六页笔记的核心知识已经掌握。

本资料为学习用校订版。Python 是 Python Software Foundation 的商标。
题目与解析为原创编写；外部链接仅用于知识核验。